第2章 信息的表示和处理 (考点, IEEE)

寻址

2.1 信息存储

寻址和字节顺序：大端法(高位存在低地址)，小端法(低位存在低地址)

十进制数字x 的ASCII码正好是 0x3x.

位运算 & | ~ ^

△如何不用第3变量，只用指针交换两数？(书P78)

```
*y = *x ^ *y    *x = *x ^ *y    *y = *x ^ *y.
```

逻辑运算 && || ! 非0即1.

移位运算. 算术移位补符号，逻辑移位补0.

sal / sar　　shl / shr.

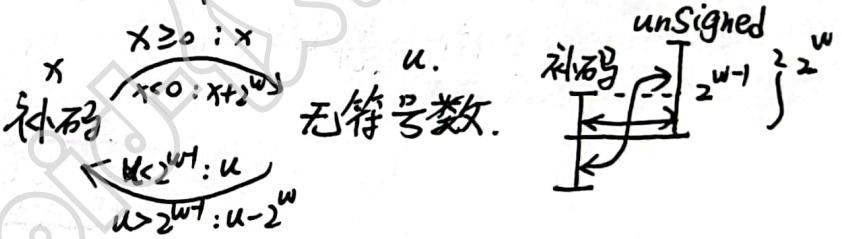
(加减法优先级比移位高，加括号！)

2.2 整数表示

无符号数编码 $B2U_w(\vec{x}) \doteq \sum_{i=0}^{w-1} x_i \cdot 2^i$

补码编码 $B2T_w(\vec{x}) \doteq \sum_{i=0}^{w-1} x_i \cdot 2^i - x_{w-1} \cdot 2^{w-1}$ (最高位负权)

补码 ⟺ 无符号数



零扩展与符号扩展.

截断：截断无符号数 $x' = x \bmod 2^k$

截断补码 $x' = U2T_k(x \bmod 2^k)$ (先当无符号，再转为补)

2.3 整数运算.

无符号数加法

无符号数求反

补码加法

补码求反(补码的非)

无符号乘法

补码乘法.

乘以常数.

除以2的幂 → 无符号数

→ 补码 → 向下舍入 $\lfloor x/2^k \rfloor = x >> k$

向上舍入 $\lceil x/2^k \rceil = (x + ((<<k) - 1)) >> k$

(向零)　　$(x + 2^k - 1) >> k$ 偏置量.

2.4 解浮点数

二进制小数 $b=\sum_{i=-n}^{m} 2^i \times b_i$ （小数部分n位，整数部分m位）

规格化浮点数 $x=(-1)^S \times (1.M) \times 2^{E-127}$ [单精度]

单精度下 S：1位 E：8位 M：23位.

双精度下 S：1位 E：11位. M：52位

~~通式~~

对于一般情况：S | E（阶码，k位） | M（尾数）.

通式 $(-1)^S \times (1.M) \times 2^{e-bias}$ $bias=2^{k-1}-1$

$E=e-bias$

规格化：E不为全0，不为全1. $E=e-bias$, $(-1)^S \times (1.尾) \times 2^E$

非规格化：E为全0 $E=1-bias$, $(-1)^S \times (0.尾) \times 2^E$

无穷大：E为全1，M为全0

NaN：E为全1，M不为全0.

△思考：一个n位（M位小数，E位阶码）定点小数，阶码是无符号，不能表示的最小正整数为：2^{2^n+1}

示例表：（正数）

非规格化数	0	0	00…0	00…0
	最小非规格化数	0	00…0	0…01
	最大非规格化数	0	00…0	1…11
规格化数	最小规格化数	0	0…01	0…00
	最大规格化数	0	1…10	1…11
∞		0	1…11	0…00

舍入，向偶舍入 上 最低有效位后紧跟1个1时，考虑使最低有效位变成0的方法.

即 X…XXXX.YY…Y|100……00

最低有效位

若最低有效位后紧接0，舍.

接1但后仍有1，入.

浮点运算.

第3章 程序的机器级表示.

△指令

```
gcc -Og -o p p1.c p2.c
(gdb) x/14xb function.
                 (14组) (1字节)
objdump -d x.o.
```

3.2 程序编码.

objdump 反汇编器. ~~gcc~~ 反汇编 .o和可执行文件的区别(链接)

ATT & Intel

内联汇编

3.3 数据格式(以第2版为准)[32位机]

C声明	Intel数据类型	后缀	大小
char	字节	b	1
short	字	w	2
int	双字	l	4
long int	双字	l	4
char*	双字	l	4
float	单精度	s	4
double	双精度	l	8
long double	扩展精度	t	10/12

3.4 访问信息.

```
%eax (32)  %ax(16)(低)  %ah(8-15)  %al(0-7)
%ecx                    (w)        (.b)
%edx
%ebx
%esi (32)  %si(16)(低)
%edi
%esp
%ebp
```

立即数$ 寄存器 R[%eax] 内存引用. M[0x8048032]

寻址模式: $Imm(r_b, r_i, s)$

Imm: 立即数; r_b: 基址reg; r_i: 变址reg; s: 比例因子 s=1,2,4,8.

寻址：立即数寻址，寄存器寻址，绝对寻址，间接寻址，变址寻址，比例变址寻址

$0x20 → 0x20

%eax → eax内是M的地址 R[%eax]

0x20 → M[0x20]

(%eax) → M[R[%eax]]

3(%eax,%ebx) → M[3+R[%eax]+R[%ebx]]

没有比例因子S 但有基址reg

比例变址寻址

3(%eax,%ebx,4) → M[3+R[%eax]+4*R[%ebx]]

有比例因子S

(基址+偏移量)寻址.

3(%eax) → M[3+R[%eax]]

只有1个reg

数据传送指令.

mov : movb movw movl

movs : movsbw movsbl movswl

movz : movzbw movzbl movzwl.

自带对%esp的操作

sub $4, %esp
mov S, (%esp)

pushl S
R[%esp] ← R[%esp] - 4
M[R[%esp]] ← S.

popl D
D ← M[R[%esp]]
R[%esp] ← R[%esp] + 4

add mov(%esp), D
add $4, %esp

3. 算术和逻辑操作. (书P129)

加载有效地址 leal S D D ← &S 不访存.

一元操作、二元操作 指令 S D → D ⊕ S.

必须先从内存中取出.

移位操作 SAL SHL SAR SHR.

(算术移位) (逻辑移位)

移位量必须用最低的字节

立即数必须加 $

如 sall %bl, %eax

△ xor %edx, %edx 实现清零.

特殊算术操作: imull mull idivl divl

(有符号) (无符号)

乘: 目标操作数预先存在%eax中.

源操作数 通过 imull S 给出

结果: 高32位 → %edx

低32位 → %eax.

然后持续至栈中.

除: 被除数: %edx(高32位) %eax(低32位)

除数: 通过 idivl S 给出.

结 商: → %eax

余数: → %edx

cltd 指令: 将%eax符号扩展至%edx

方便设定被除数.

3.6 控制.

条件码 CF, ZF, SF, OF.

CMP S_1, S_2 做($S_2 - S_1$)并改变条件码
TEST S_1, S_2 做($S_2 \& S_1$)并改变条件码
> 不改变 S_2.

set系列.

跳转指令: 无条件跳转 jmp ← 直接跳转 jmp .L
间接 jmp *(%eax) 或 jmp *%eax 等
条件跳转 (只能直接跳转)

△跳转指令编码 (第7章 链接有详细)

小端写法.

80482bf e9 e0 ff ff ff jmp ?
80482c4 90 nop

这个 + [ff ff ff e0] 回填.

怎么算是多少.

汇编代码中是一个相对量 offset

jmp 下一条指令的地址 + offset = 跳转目标地址.

△ret前的rep有什么用.

条件控制: (书P143) 转移.

条件传送 实现条件分支. (数据的条件转移).

条件传送指令 cmove族.

更快, 但有缺陷. 如 return (xp? *xp:0);

容易产生不期望的非法操作.

上次例: 若被除数<0. 必须加上偏移量 $2^k - 1$ (练习3.20)

循环.

① do-while循环 do{ }while( );

-Og

② while循环 while( ){ };

```
loop:
    .....
    t=test-expr
    if(t) goto loop.
```

```
    goto test;
loop:
    .....
test:
    t=test-expr
    if(t) goto loop.
```

-O1

```
    t=test-expr
    if (!t) goto done;
loop:
    t=test-expr
    if(t) goto loop;
```

(3) for 循环. for(init;test;update){...}; init while(){ update }

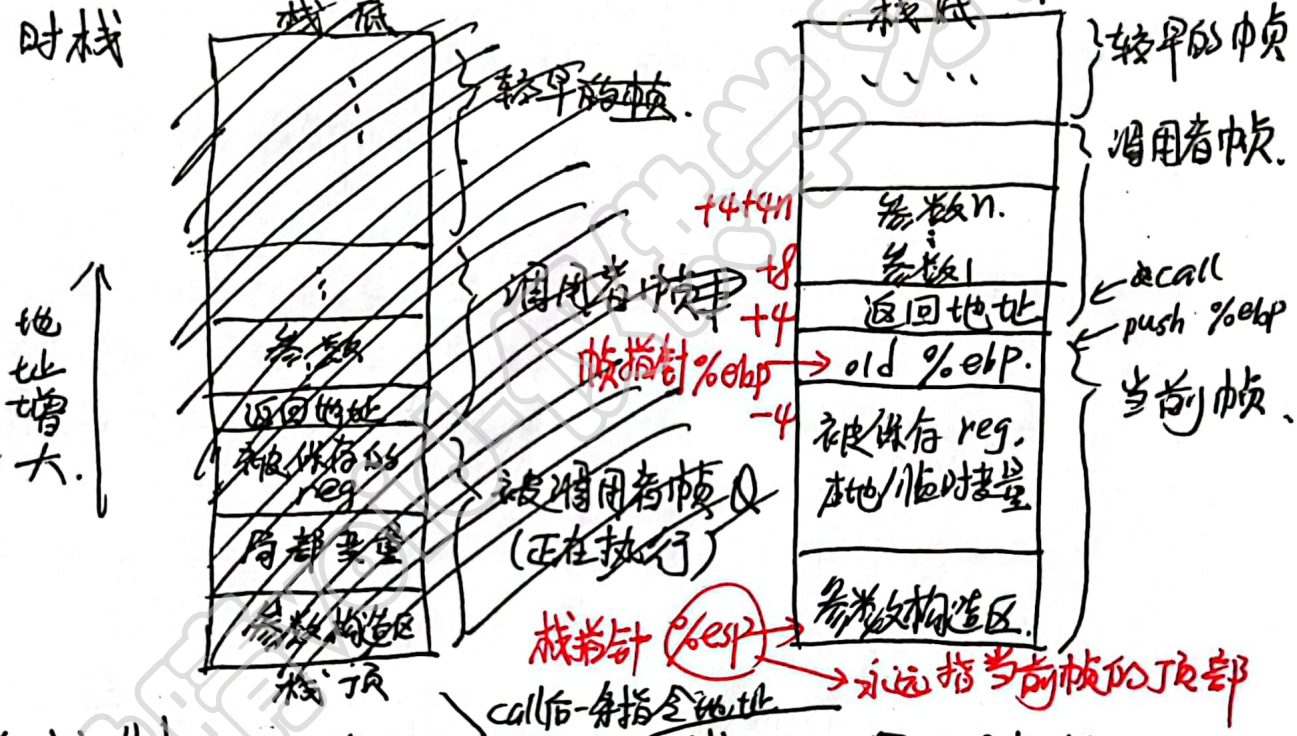
△ continue存在时 for循环转换while循环 要注意 update部分. 可用goto新标签替代.

switch语句 . 跳转表. (练习典型题 P163)

3.] 过程 [难点★] (以第2版为准) 这里32位与64位有很大不同.

传递控制(程序计数器PC)、传递数据、分配&和释放内存.

运行时栈



转移控制 call: 返回地址压栈, PC设置为Q起始.

ret 弹出返回地址, PC设置为返回地址.

leave 使栈做好返回准备 ⇔ movl %ebp, %esp
popl %ebp. 恢复old ebp

寄存器使用惯例(约定如此) 调用者保存reg: %eax, %edx, %ecx.

被调用者保存reg: %ebx, %esi, %edi.

即对于被调用者P来说是否可无代价覆盖使用

△GCC如Q不使用的空间→对齐, 一般被使用为16的整数倍.

3.8 数组的分配与访问

连续的空间，访问 起始(,i,4) 即为 a[i] (int型)

指针运算.

嵌套的数组 &D[i][j] = Xb + L(c·i+j) (定义D[][c])

定长数组.

变长数组.

3.9 异质的数据结构.

结构

联合：联合大小取其中最大的

数据对齐：Linux要求 short地址必须为2的倍数.
int, int*, float, double 地址须为4的倍数.

3.10 综合理解指针.

3.11 GDB

3.12 存储器越界与缓冲区溢出.

对抗缓冲区溢出攻击的方法.

栈随机化 栈破坏检测 限制可执行代码区域

第5章 优化程序性能（以PPT为索引）

5.1. 优化编译器的能力及局限性

5.2 表示程序性能：CPE（Cycles per Element）每元素的周期数

→ 优化改进思路（软件部分）

① 消除循环低效率：每次循环都要算的重复过程可取出来

② 减少过程调用.

③ 减少访存：采用局部变量存储（会保存在reg里）

~~→ 优化改进思路（硬件部分）~~

→ 指令流水线基础.

取指令 → 指令译码 → 取操作数 → 执行 → 写回.

指令级并行：指令间本不存在相关性.

结构相关：不同指令同时取相同的reg/存储器，但无数据流.

数据相关：该指令操作数依赖前几条指令的结果. 数据与指令分开，避免访存冲突

写后读，读后写，写后写 数据重排，降低数据依赖

控制相关：分支预测 降低预测错误概率.

→ 理解现代处理器.

① 循环展开：减少循环迭代次数，一个循环内算多次.

② 多路并行：使用2个累积变量（分配到不同寄存器），每次迭代中可并行处理

③ 重新结合变换：减少了关键路径.

执行时间下限：关键路径、功能单元吞吐量 共同决定.

△ Amdahl定律：系统优化的通用原则.

加快一个系统一个部分的速度，对整体性能影响依赖于

该部件多重要（α），该部件速度提高了多少（k倍）.

~~方法比~~ $T_{new} = (1-\alpha)T_{old} + \frac{\alpha T_{old}}{k} = T_{old}[(1-\alpha)+\alpha/k]$

故 加速比 $S = \frac{1}{1-\alpha+\alpha/k}$.

△ 对图会看即可（不要求会改会画）.

第7章 链接

7.1 编译器驱动程序.

main.c —cpp（预处理器）→ main.i —cc1（C编译器）→ main.s（汇编文件）—as（汇编器）→ main.o（可重定位目标文件）—ld（链接器）→ 可执行文件.

若要编译执行该文件 Linux> gcc -Og -o prog main.c sum.c

分步执行 Linux> cpp main.c main.i

Linux> cc1 main.i -Og -o main.s

Linux> as main.s -o main.o

Linux> ld main.o sum.o -o prog

7.2 静态链接

符号解析 & 重定位：
1. 将各自代码段与数据段分别合并
2. 将.o文件中符号的相对位置重定位到相应的绝对存储位置.
3. 更新符号引用到绝对位置

（重定位节和符号定义 / 将符号引用与相应的符号定义关联）

重定位节中的符号引用.

7.3 目标文件

区别：

- 可重定位目标文件 .o file
- 可执行目标文件 .out file
- 共享目标文件 .so file

7.4 可重定位目标文件. (书P467)

.text .rodata .data .bss .symtab .rel.text .rel.data .debug .line .strtab

已(未)初始化的全局和静态 C 变量存储在哪，初始化为0吗？局部变量呢？

7.5 符号和符号表

- Global 全局符号：全局变量，非static的函数
- External 外部符号：其它模块中的全局变量，非static的函数.
- Local 本地符号：只被该模块定义的static函数/static全局变量.

对本地未初始化 temp（函数内）不关心 ← 局部变量在栈上，不进符号表.

7.6 符号解析.

强符号：函数/已初始化的全局变量

弱符号：未初始化的全局变量.

规则：① 强符号只可有1个，否则error

② 有强符号1个，弱符号多个，选强舍弱（被全部替换）

③ 有弱符号多个，任意选择（不告知用户）.

一个例子，若A中 int x=5, y=5; B中 double x; 则修改B中x时，会影响A中的x和y.

△静态库链接.

创建静态库 ar rcs libvector.a addvec.o multvec.o.

使用静态库 gcc -static (-o a.out) main.o ./libvector.a ← 推荐.
或 -L. -lvector

只会从静态库中取需要的部分，未引用到的不取.

△静态库解析引用（链接器的工作） (书P477)

集合E, U, D

从左至右依次读取. 若为目标文件，则添至E，无法解析的成员放至U.
若为库文件，则使用该文件解析U，若有U中可被解析，就放到E.

一遍读取完后，若 U≠∅，则报错.

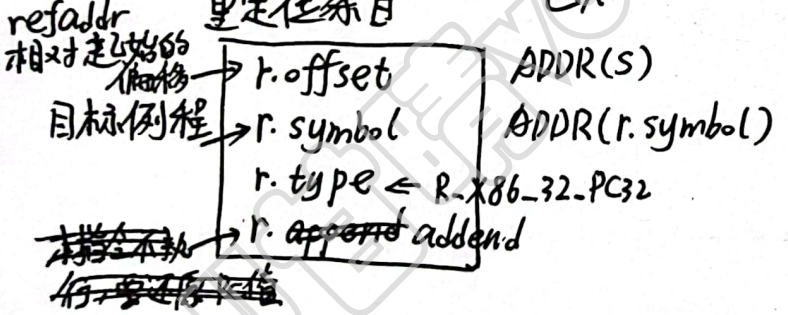
△库一般放最后，若有交叉错情况，可出现多次库.

7.7 重定位. ☆

☆重定位要完成什么(书P479, 478)？重定位节和符号定义，重定位节中的符号引用

重定位有2种，重定位PC相对引用、重定位绝对引用

①重定位 PC相对☆引用



~~持续不执行，要重新计算~~

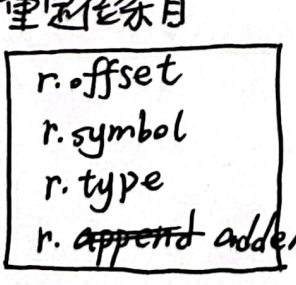
计算

①引用(call后)的运行时地址.
refaddr = ADDR(s) + r.offset.

②更新该地址内容.
*refaddr = ADDR(r.symbol) + r.addend - refaddr.

③执行时：PC保存后，PC ← PC + *refaddr

②重定位绝对引用.



计算

①~~将该地址~~引用的运行时地址.
refaddr = ADDR(s) + r.offset.

②更新该地址内容
*refaddr = ADDR(r.symbol) + r.addend

☆注意小端及写法，不要出错.

问重定位引用地址是多少？(refaddr) 问重定位引用值是多少？(*refaddr)

7.8 可执行目标文件.

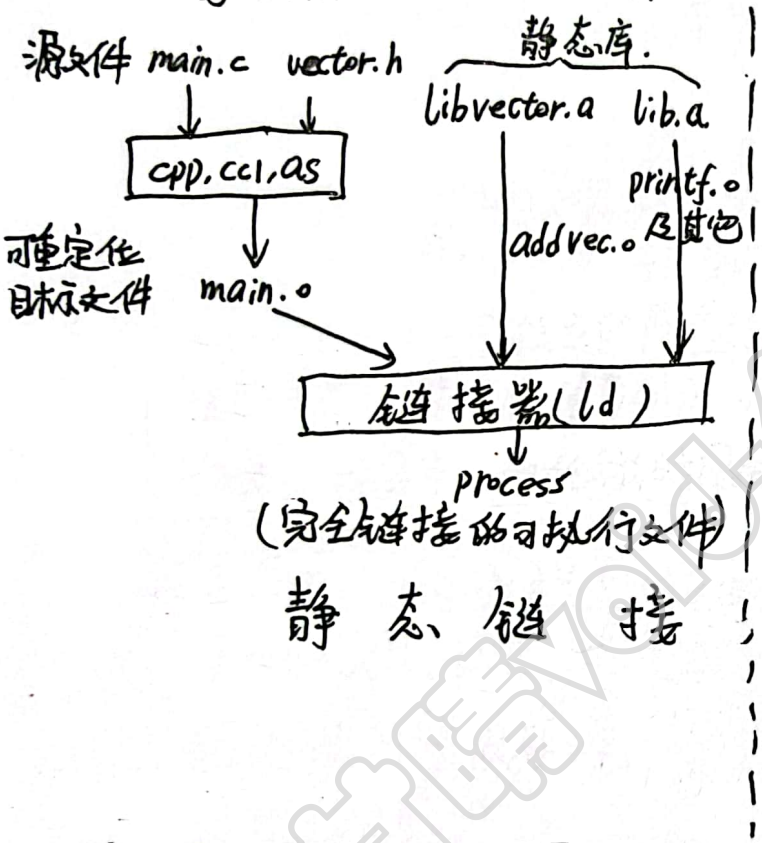
* 与可重定位目标文件的区别:
  所有节都已经被重定位到最终运行时内存地址.
  程序入口点(.init节定义函数_init

7.9 加载可执行目标文件.

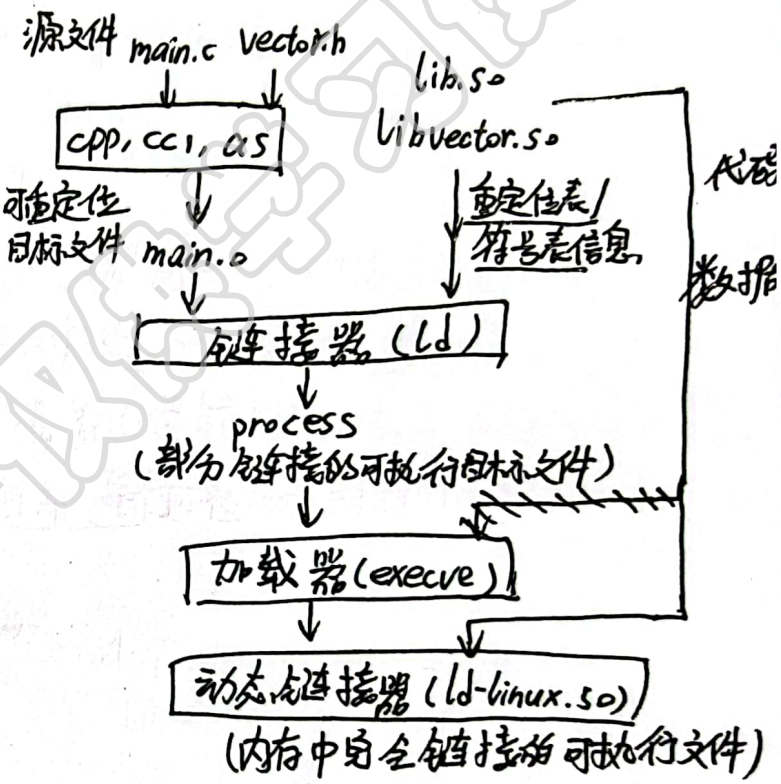
(OS) 父shell fork 一个子shell, 子shell 通过 execve() 启动加载器.

7.10 动态链接共享库 (Linux中采用.so表示, windows下用 DLL表示)

静态链接 <> 动态链接.



静态链接



动态链接

指令 Linux > gcc -shared -fpic -o libvector.so addvec.c multvec.c
Linux > gcc -o process main.c ./libvector.so.

7.11 从应用程序中加载和链接共享库.

7.12 PIC位置无关代码 (GOT与PLT). Position-Independent-Code.

第8章 异常控制流 (ECF)
Exceptional Contrd flow

8.1 异常

异常处理 异常号(非负整数) 异常表(OS:陷阱表) 异常表基址寄存器.
异常<>过程调用(书P503).
异常的类别:4种,以及处理方式,返回行为(书P504)
中断(异步发生,返回至next指令继续执行)
陷阱&系统调用(同步发生,返回至next指令)
故障(处理成功→返回原指令;处理失败→终止) e.g.缺页.
终止.

8.2 进程(定义:一个执行中程序的实例)

上下文:栈,通用reg,程序计数器PC,环境变量,文件描述符打开集合……
关键抽象:一个独立的逻辑控制流,一个私有的地址空间.
逻辑控制流(PC值序列) 并发流(逻辑控制流在执行时间上重叠)
并发 时间片 多任务(时间分片) 并行流(并发地运行在多核或多机上)(书P509)
私有地址空间
用户模式与内核模式(模式位)
上下文切换,调度,调度器.

8.3 系统调用错误处理

8.4 进程控制

获取进程ID: getpid() getppid()
创建与终止进程 fork() 创建1次返回2次;并发执行;相同但独立空间;共享文件.
终止:①信号 ②从主程序返回 ③调用exit()(不返回)
回收子进程: 僵尸进程(已死未收),孤儿进程(init进程成为养父)

```
pid_t waitpid(pid_t pid, int *statusp, int options);
```

相关参数定义及宏见(书P517)!

让进程休眠: sleep(), pause() (书P521)
加载并运行程序: execve() 除非出错,从不返回.
利用fork与execve运行程序 (Shell-Lab)

6

存储器层次结构.

书P421

CPU寄存器 (编译器管理)

高速缓存 L1, L2, L3 (SRAM) 硬件

主存 (DRAM).

磁盘

6.1 存储技术.

RAM < SRAM.(静态) Cache.
DRAM.(动态). 主存

TLB 硬件 MMU 管理

磁盘: 磁盘容量 = (扇区字节数) × (每磁道扇区数) × (每面磁道数) × (盘面数) × (盘片数)

读写头一致行动.

寻道. 旋转. 传输 (OS).

固态硬盘

6.2 局部性 ☆

时间局部性 — 数据引用

空间局部性 — 取指令.

6.3 存储器层次结构

缓存 — 命中

不命中 — 强制 (冷)

冲突 (空间足够但冲突)

容量 (工作集 > 缓存大小)

6.4 Cache. ☆

1 通用: $M=2^m$ 个地址.

组数 $S=2^s$ 块大小 $B=2^b$.

t + s + b = m

标记 CT | 组索引 CI | 块偏移 CO.

B (块大小)

S组 { E行 (也叫相联度)

一般以"块"为单位

称作高速缓存 (S, E, B, m)

高速缓存大小 $C=B\times S\times E\times B$.

2 直接映射高速缓存 (E=1) 组选择 → 行匹配 (行=1) → 字抽取

冲突不命中 (P431) 抖动: 高速缓存反复地加载和驱逐相同的 Cache 组. (解决: 在数组结尾故填充)

为什么用中间位作索引 (P432) 若用高位作索引: 地址上连续的块会被映射到相同的组, 产生大量冲突不命中.

3. 组相联 Cache. (E>1) X路组相联

4. 全相联 Cache. (S=1) 无组索引 CI.

CT | CO.

(只适合做 TLB) 看书评论明缓冲器

5. 有关写的问题 — 直写

写回 (要 dirty bit 脏位) — 通常: 写非写分配 / 写分配 (写加载再更新)

6. i-cache/d-cache.

6.5 高速缓存友好代码.

给制命中准则. 计算命中率/不命中率 (P442-443)

① 重点关注 高速缓存Cache能装下它用的多少.

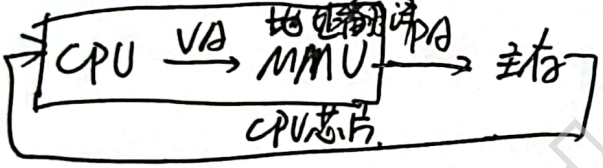
② 举例看一部分的情况 ③ 认为投位置与被访性表此人

④ 冷不命中不可避免.

6.6. Cache对性能的影响.

第9章 虚拟内存.

9.1 ~~物理和虚~~ PA & VA



9.2 地址空间.

VA: $N=2^n$ PA: $M=2^m$

9.3 ~~主~~虚拟内存VM 虚拟页面 {未分配的 / 分配的 {缓存的 / 未缓存的} ← 由valid有效位确定

页表: 虚拟页 —映射→ 物理页.

PT (可能存在物理页上)

页表条目. 内存

由PTE (page table entry) 组成.

页命中 在PTE中 VP有效位为1. 故VP被缓存在DRAM中.

缺页 在PTE中 VP有效位为0. → 触发缺页异常 → 物理内存页.

处理完后将继续翻译. P564.

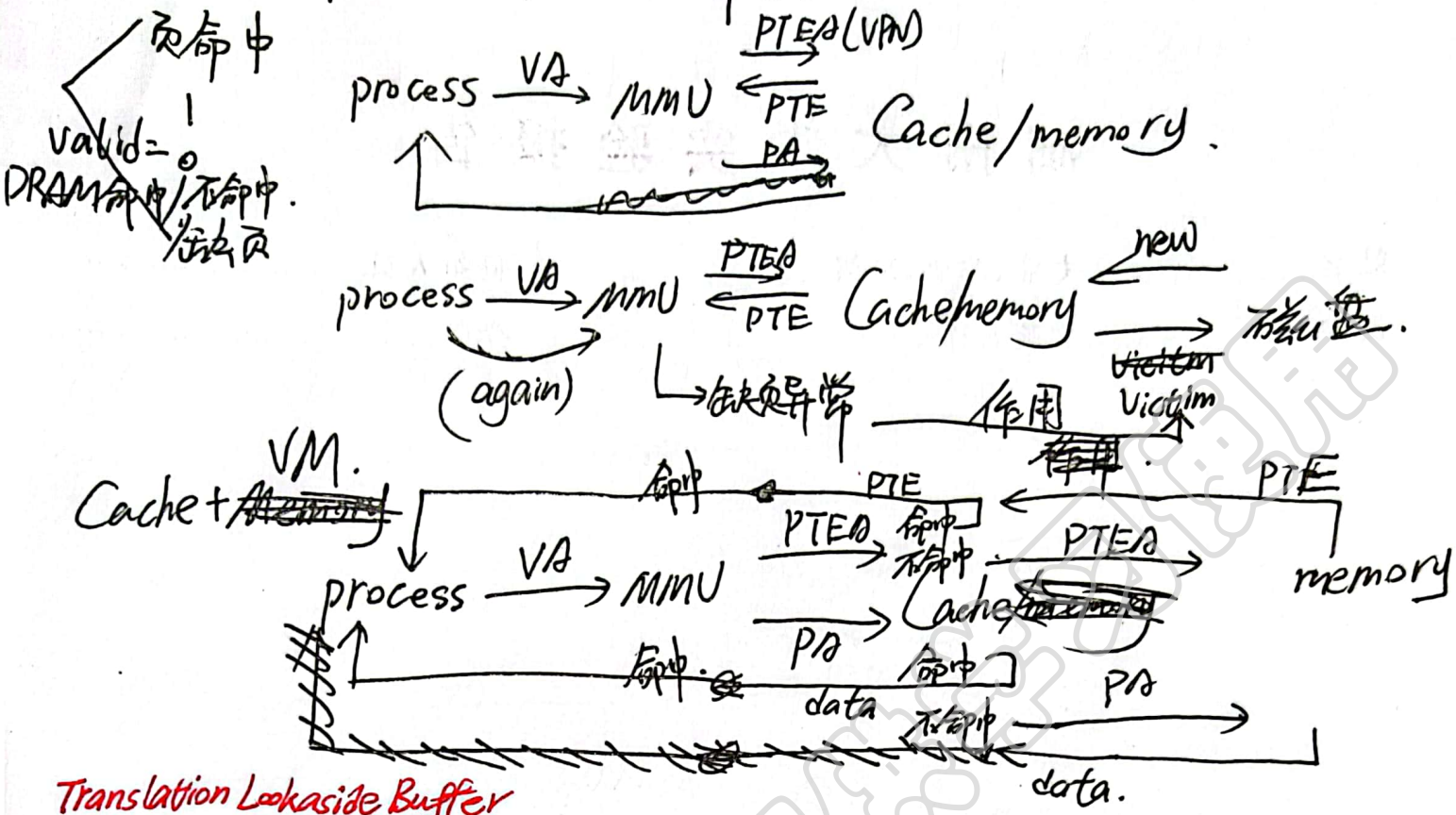
未分配页面 DRAM缓存不命中

9.4 VM作为memory管理工具.

9.5 VM作为memory保护工具: PTE上添加许可位 控制对VP的访问.

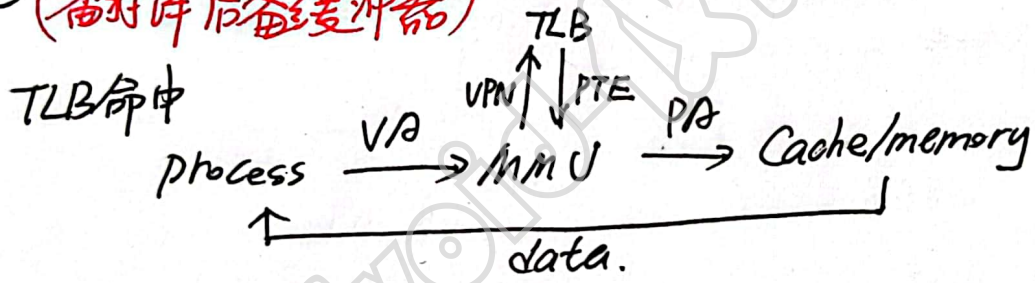
9.6 地址翻译 ☆

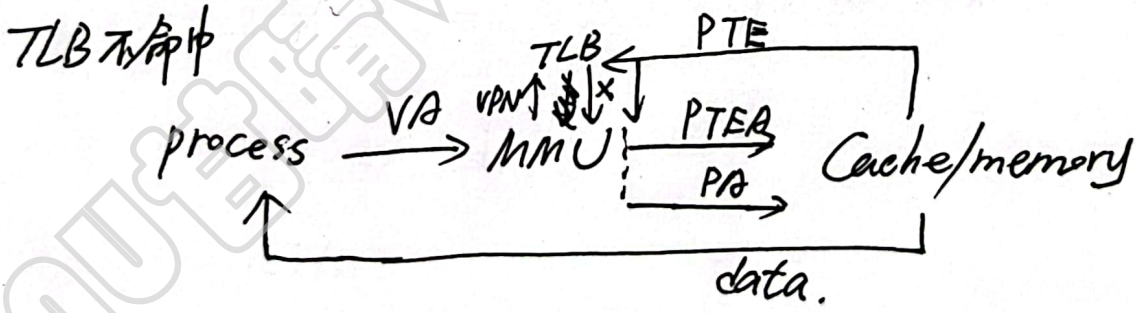
这部分主要2个点：流程 + 翻译过程.



Translation Lookaside Buffer

TLB (翻译后备缓冲器)



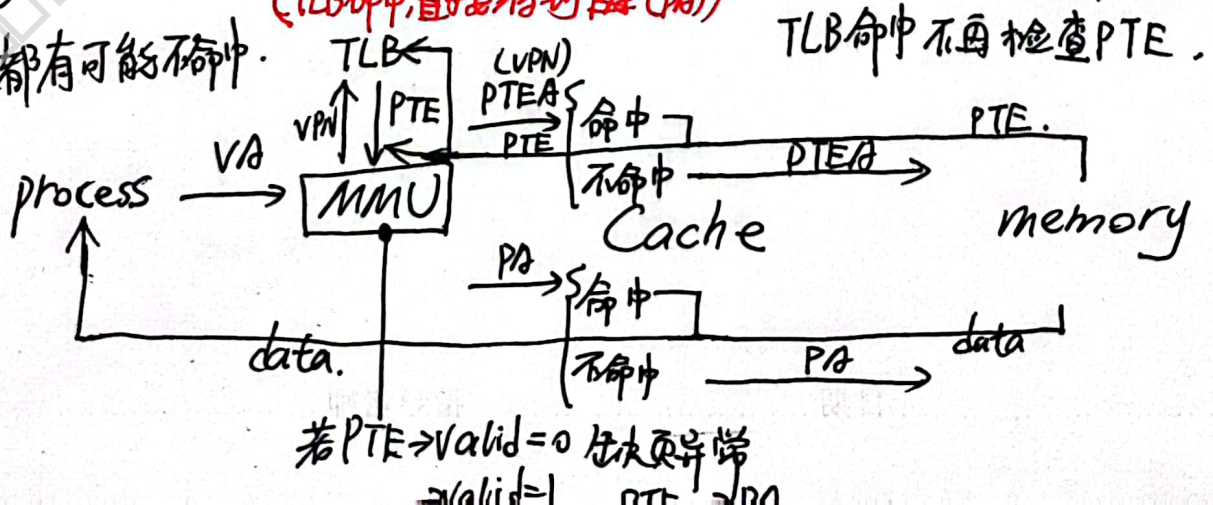


综合：

每一级都有可能不命中.

(TLB命中,直接得到 PPN (PA))

TLB命中不再检查PTE.



多级页表. 多层次.

考题都是这么出.

地址翻译. $N=2^n$ (virtual), $M=2^m$ (pysical), $P=2^p$ TLB.

TLB: 四路组相连, 16个条目.
↳ TLBI=2 (4组).

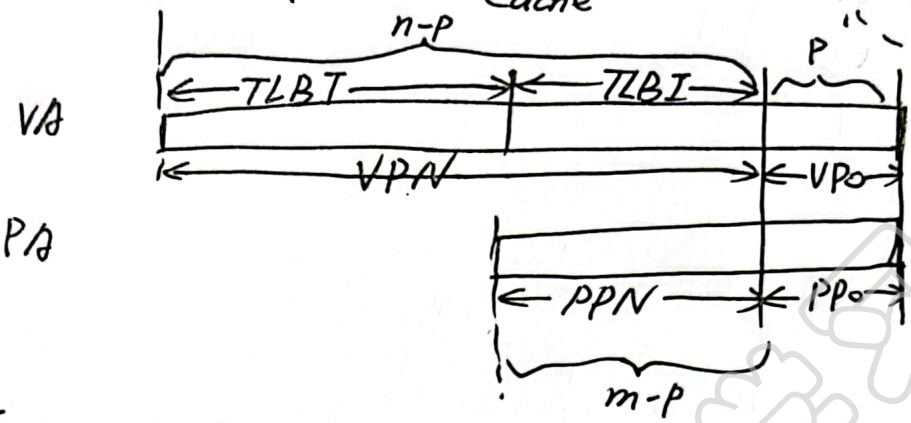
书P573

VA: VPO_{n-p} VPN_{n-p} TLBI TLBT

PA: PPO_{m-p} PPN_{m-p} CO CI CT

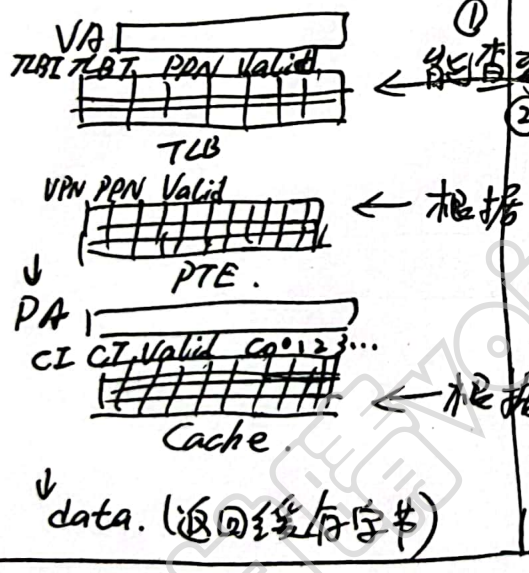
Cache: b, s, t

$b+s+t=m$
$C=B\times E\times S$
$2^b\times E\times 2^s$



典型题:

给定VA



① 能查到就查到. ③ 查不到就是"TLB不命中"

② 查到但Valid=0 → "TLB不命中".

② 根据VPN查PTE, 发现Valid=0 → "缺页".

① 查到 (不可能查不到)

① 根据PA查, 有就返回data. ② 无就是"缓存不命中".

9.7 案例 (icore i7.

9.8 内存映射.

9.9 动态内存分配

9.10 垃圾收集

9.11 与内存有关错误